National University of Computer and Emerging Sciences

Karachi Campus

Data Structures (CS2001)

Date: 15th Dec 2025

Course Instructor(s)

Dr. Jawwad, Dr. Anam, Dr. Farrukh, Mr. Basit, Mr. Farooq, Mr. Nouman, Ms. Sania, Ms. Fizza, Ms. Mubashara

Do not write below this line

Attempt all the questions.

CLO # 2: *Solve* recursive problems efficiently using Backtracking

Roll No _____ Section _____

Student Signature _____

Final Exam

Total Time (Hrs): 3

Total Marks: 50

Total Questions: 7

Q1: Given a singly linked list that may or may not contain a cycle. You are required to write a C++ function that do the following: **[7 Marks]**

a) Detect whether a cycle exists in the list.
b) If a cycle is present, identify the node where the cycle begins.
c) Remove the cycle completely by modifying the next pointer of the node that forms the cycle.
d) Return the head of the corrected linear linked list.

Solution:

```
struct Node {
   int data;
   Node* next;
};

Node* removeCycle(Node* head) {
   Node* curr = head;

   while (curr != NULL) {
      Node* check = head;

      // check if curr->next points to any previous node
      while (check != curr) {
         if (curr->next == check) {
            // cycle detected
            curr->next = NULL;  // remove cycle
```

```
            return head;
        }
        check = check->next;
      }
      curr = curr->next;
    }
    return head;
}
```

CLO # 3: *Compare* different data structures in terms of their relative efficiency and *design* effective solutions and algorithms that make use of them.

Q2: You are given a binary family tree where each node represents a family member, and each level represents a generation. Write a C++ function that finds the Most Recent Common Ancestor (MRCA) of all members in the deepest generation of the tree. **[7 Marks]**

1. First, identify all the nodes that appear at the maximum depth.
2. If there is only one such node, return that node.
3. If there are multiple deepest nodes, return their Lowest Common Ancestor (LCA).

Your function should take the root of the tree and return the family member who is the MRCA of all nodes in the deepest generation.

The Lowest Common Ancestor (LCA) of two nodes in a binary tree is the deepest (or lowest) node in the tree that has both nodes as descendants.

- Deepest Nodes: 8 and 50
- MRCA: 20

Solution:

```
#include <iostream>
using namespace std;
class Node {
public:
int data;
Node* left;
Node* right;
Node(int val) {
data = val;
left = right = nullptr;
}
};
// Function to compute maximum depth of tree
int maxDepth(Node* root) {
if (root == nullptr)
return 0;
int leftDepth = maxDepth(root->left);
int rightDepth = maxDepth(root->right);
return 1 + max(leftDepth, rightDepth);
}
// Function to find LCA of all deepest leaves
Node* findMRCA(Node* root, int depth) {
if (root == nullptr)
```

```
return nullptr;
// If we reached the deepest level, return node itself
if (depth == 1)
return root;
Node* left = findMRCA(root->left, depth - 1);
Node* right = findMRCA(root->right, depth - 1);
// If both left and right deepest nodes found → root is their ancestor
if (left != nullptr && right != nullptr)
return root;
// Otherwise return whichever side has deepest nodes

return (left != nullptr) ? left : right;
}
// Wrapper function
Node* lcaDeepestLeaves(Node* root) {
int depth = maxDepth(root); // find max tree depth
return findMRCA(root, depth); // find MRCA of deepest nodes
}
int main() {
/*
1
/ \
2 3
/ / \
4 5 6
/
7
*/
Node* root = new Node(1);
root->left = new Node(2);
root->right = new Node(3);
root->left->left = new Node(4);
root->right->left = new Node(5);
root->right->right = new Node(6);
root->right->left->left = new Node(7);
Node* ans = lcaDeepestLeaves(root);
cout << "MRCA of deepest generation = " << ans->data << endl;
return 0;
}
```

CLO # 3: *Compare* different data structures in terms of their relative efficiency and *design* effective solutions and algorithms that make use of them.

Q3: Patients arrive at a hospital Emergency Room (ER). Each patient has a severity score. The hospital treats one patient at a time, using the following rules: **[7 Marks]**

- The patient with the highest severity is treated first.
- If two or more patients have the same severity, the one who arrives earlier is treated first.

You must implement the C++ function: **void simulateER(int* severity, int n, int* treatmentOrder);**

- severity → an array of severity scores of the patients **in arrival order**
- n → total number of patients

- treatmentOrder → an output array where you will store the order in which patients are treated. You must store the **patient indices** (0 to n-1), where each index refers to the position of that patient in the severity array.

You must simulate the ER using:

- a **Max-Heap** to choose the patient with the highest severity
- a **Queue** to break ties when severities are equal

```
/ class to store patient information
class Patient {
   int severity;
   int index;   // arrival order
};

// Swap two patients
void swap(Patient &a, Patient &b) {
   Patient temp = a;
   a = b;
   b = temp;
}

// Heapify function for Max-Heap
void heapify(Patient heap[], int size, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   // Compare left child
   if (left < size) {
      if (heap[left].severity > heap[largest].severity ||
        (heap[left].severity == heap[largest].severity &&
         heap[left].index < heap[largest].index)) {
         largest = left;
      }
   }

   // Compare right child
   if (right < size) {
      if (heap[right].severity > heap[largest].severity ||
        (heap[right].severity == heap[largest].severity &&
         heap[right].index < heap[largest].index)) {
         largest = right;
      }
   }

   // If largest is not root
   if (largest != i) {
      swap(heap[i], heap[largest]);
      heapify(heap, size, largest);
   }
}
```

```
// Build Max-Heap
void buildHeap(Patient heap[], int n) {
   for (int i = n / 2 - 1; i >= 0; i--) {
      heapify(heap, n, i);
   }
}

// Function to simulate ER
void simulateER(int* severity, int n, int* treatmentOrder) {
   // Create heap array
   Patient* heap = new Patient[n];

   // Insert patients into heap
   for (int i = 0; i < n; i++) {
      heap[i].severity = severity[i];
      heap[i].index = i;   // arrival order
   }

   // Build Max-Heap
   buildHeap(heap, n);

   int heapSize = n;
   int order = 0;

   // Extract patients one by one
   while (heapSize > 0) {
      // Root has highest priority patient
      treatmentOrder[order++] = heap[0].index;

      // Move last element to root
      heap[0] = heap[heapSize - 1];
      heapSize--;

      // Restore heap property
      heapify(heap, heapSize, 0);
   }

   delete[] heap;
}
```

CLO # 3: *Compare* different data structures in terms of their relative efficiency and *design* effective solutions and algorithms that make use of them.

Q4: Consider that the social media platform processes thousands of hashtags daily, and you need to group all hashtags that are anagrams of each other for trend analysis. Two hashtags are considered anagrams if they contain the same letters in any order (ignore case if needed). As many different hashtags can share the same anagram signature, the system must handle heavy collisions efficiently, keep insertions fast, and preserve the original posting order. The team decided to use a custom hash table with separate chaining.

[2+2+4 Marks]

a) Explain why separate chaining is more suitable than open addressing for this anagram-grouping scenario.
Heavy Collisions Handling, Fast Insertions, and Order preservation.

b) How the hash key should be generated, so that anagrams map to the same chain?
There can be multiple answers, One possible answer is:
Convert the hashtag to lowercase (to ignore case).
Sort the characters of the hashtag.
Use the sorted string as the hash key.

Example
Input:
["fun", "nuf", "code", "deco", "node"]
Output:
["fun", "nuf"]
["code", "deco"]
["node"]

c) Provide a C++ code that builds a separate-chaining hash table, groups anagram hashtags, and maintains the order of appearance.

```
using namespace std;

const int TABLE_SIZE = 10;

// Linked List Node
class Node {
public:
   string key;      // sorted anagram key
   string hashtag;  // original hashtag
   Node* next;
};

// Hash Table (array of linked lists)
Node* hashTable[TABLE_SIZE];

// Initialize hash table
void initHashTable() {
   for (int i = 0; i < TABLE_SIZE; i++)
      hashTable[i] = NULL;
}

// Generate anagram key
string generateKey(string s) {
   for (int i = 0; i < s.length(); i++)
      s[i] = tolower(s[i]);
   sort(s.begin(), s.end());
   return s;
}

// Hash function
int hashFunction(string key) {
   int sum = 0;
   for (int i = 0; i < key.length(); i++)
      sum += key[i];
```

```
    return sum % TABLE_SIZE;
}

// Insert into hash table (preserves order)
void insert(string hashtag) {
    string key = generateKey(hashtag);
    int index = hashFunction(key);

    Node* newNode = new Node;
    newNode->key = key;
    newNode->hashtag = hashtag;
    newNode->next = NULL;

    // If bucket is empty
    if (hashTable[index] == NULL) {
        hashTable[index] = newNode;
    }
    else {
        Node* temp = hashTable[index];
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;   // append to maintain order
    }
}

// Display grouped anagrams
void display() {
    for (int i = 0; i < TABLE_SIZE; i++) {
        if (hashTable[i] != NULL) {
            cout << "Bucket " << i << ": ";
            Node* temp = hashTable[i];
            while (temp != NULL) {
                cout << temp->hashtag << " ";
                temp = temp->next;
            }
            cout << endl;
        }
    }
}
```

CLO # 1: *Use & explain* concepts related to basic and advanced data structures and describe their usage in terms of common algorithmic operations

Q5: Show trace (dry-run), how shell sort sorts the array "E A S Y S H E L L S". Use gap = n/2. Give the total number of exchanges taken by shell sort to sort the given array. **[7 Marks]**

SOLUTION

Gap sequence: n/2 , n/4 , ..., 1
n = 10, so gaps = 5, 2, 1

0 1 2 3 4 5 6 7 8 9
E A S Y S H E L L S

GAP = 5
(2,7): S – L swap
(3,8): Y – L swap

Exchanges: 2

5-sorted
0 1 2 3 4 5 6 7 8 9
E A L L S H E S Y S

GAP = 2
Compare S & L swap
Compare L & E swap
Compare S & E swap
Compare L & A swap
Compare Y & S swap

Exchanges in this pass: 5
Total so far: 7

2-sorted
0 1 2 3 4 5 6 7 8 9
E A E L L H S S S Y

Now with gap = 1
E & A swap
L & E swap
H & L swap
S & H swap
Exchanges in this pass: 4
Total = 7+4= 11

CLO # 3: *Compare* different data structures in terms of their relative efficiency and *design* effective solutions and algorithms that make use of them.

Q6: You are building a search feature in a text editor. The editor allows users to search for patterns in documents efficiently. You are asked to analyze one search query using two different string matching algorithms: KMP and Boyer-Moore (bad character heuristic only) **[4+3 Marks]**

a) Compare the behavior of the two algorithms on ***Text:*** **ABABACABAABABAC** & ***Pattern:*** **ABABAC**. Which algorithm performs minimum character comparisons? Show all the steps clearly.

KMP Approach:

LPS :

0	1	2	3	4	5
0	0	1	2	3	0

Comparisons: 16

Pattern Found: Index 0 and 9

Boyer Moore Approach:

Bad Match Table:

Using formula: length-index-1

A=1, B=2, C=6, *=6

Shifts using formula : d1=max{value of mismatched character from BMT- value of matched characters, 1}

Comparisons: 14

In this case, both algorithm wll perform almost equal. If we check the number of comparisons, Boyer moore comparisons are lesser.

b) Under which conditions, KMP outperforms?

in the worst case, such as AAAAAAAAAAAAAA is text and AAAA is pattern, in this case, KMP will outperform.

CLO # 4: *Transform* cycling-bearing graphs into acyclic tree structures for minimum cost traversal

Q7: Consider the weighted graph with adjacency lists: **[2+2+3 Marks]**

A: (B,4), (C,3), (D,1)
B: (A,4), (C,1), (D,2)
C: (A,3), (B,1), (D,4), (E,2)
D: (A,1), (B,2), (C,4), (E,3)
E: (C,2), (D,3)

a) Write the adjacency matrix for the graph. Show rows and columns in the order: A, B, C, D and E.

	A	B	C	D	E
A	0	4	3	1	0
B	4	0	1	2	0
C	3	1	0	4	2
D	1	2	4	0	3
E	0	0	2	3	0

b) Perform a Depth-First Search (DFS) starting from node A with the following tie/choice rule: when selecting which neighbor to visit next, always choose the incident edge with the minimum weight (if

two neighbors have the same edge weight, break ties by alphabetical order of node labels). Show the exact traversal order and the stack (or recursion) steps you follow.

Traversal: A → D → B → C → E

Stack:
DFS(A)
└── DFS(D)
  └── DFS(B)
    └── DFS(C)
      └── DFS(E)

c) Dry-run Prim's algorithm starting from node A to find a Minimum Spanning Tree (MST). At each step, state which edge is selected (with its weight), the set of vertices currently in the tree, and the final MST edges and total weight.

Edges Selected:

A–D (1)

D–B (2)

B–C (1)

C–E (2)

Total Weight:

1 + 2 + 1 + 2 = 6

---------- **The End** ----------